

Clinic Management System

Technical Evaluation Report

Item	Details
Report date	16 June 2026
Purpose	Detailed technical reference for evaluation — deployment, architecture, servers, frontend, and AI/ML models (inputs → outputs).
Live domain	clinic.nalexustechnologies.com
Basis	Compiled from direct inspection of the clinic_backend (Flask) and clinic_app (Flutter) source code.

1. Deployment & Domain

Item	Value
Public domain (live)	https://clinic.nalexustechnologies.com
Public API base URL	https://clinic.nalexustechnologies.com/api
Internal app URL	http://127.0.0.1:5110
App port	5110
Host OS (production)	Ubuntu / Linux
Server deploy path	/home/nalexus/clinic/clinic_backend

Correction: The earlier plain-language PDF listed the live site as restaurant.nalexustechnologies.com. That is stale/incorrect — every reference in the actual code (Flutter base URL, backend CORS allow-list, README) uses **clinic.nalexustechnologies.com**. Use the clinic domain for evaluation.

Topology: Flutter web client (browser) → HTTPS to clinic.nalexustechnologies.com → reverse proxy (TLS termination) → Gunicorn on 0.0.0.0:5110 → Flask app → MySQL. The scikit-learn risk models run in-process; the clinical LLM calls the OpenAI API (with a local fallback).

2. Backend Architecture

2.1 Stack

Layer	Technology	Version
Language	Python	3.10+
Web framework	Flask	3.0.3
WSGI server (prod)	Gunicorn (-w 4 -b 0.0.0.0:5110)	22.0.0
ORM	Flask-SQLAlchemy	3.1.1
Migrations	Flask-Migrate (Alembic)	4.0.7
Auth	Flask-JWT-Extended	4.6.0
CORS	Flask-CORS	4.0.1
DB driver	PyMySQL	1.1.1
Database	MySQL (utf8mb4)	8.0+
ML	scikit-learn / numpy	sklearn 1.5.x / numpy 1.26+

2.2 Application pattern — App Factory + Blueprints

create_app() builds the app, initializes extensions (db, jwt, migrate), enables CORS for /api/*, and registers **18 blueprints**, each mounted under an /api/... prefix:

```
/api/auth · /api/clinics · /api/super-admin · /api/clinic-admin · /api/departments · /api/doctors  
· /api/receptionists · /api/pharmacy · /api/patients · /api/appointments · /api/payments ·  
/api/assistants · /api/assistant · /api/prescriptions · /api/reports · /api/clinical-ai ·  
/api/subscriptions · /api/audit-logs · /api (health)
```

2.3 Layered design

```
routes/ → HTTP layer (blueprints, JWT guards)  
services/ → business logic (19 service modules, one per domain)  
models/ → SQLAlchemy ORM entities (18 tables)
```

```
utils/ → response envelope, password hashing
extensions/ → db, jwt, migrate singletons
```

Routes are thin; all logic lives in services. Models (18): user, clinic, department, doctor, assistant, patient, patient_vitals, patient_report, appointment, consultation_draft, prescription, prescription_lab_test, payment, pharmacy, subscription, audit_log.

2.4 Authentication & authorization

- JWT (Bearer header); access token 24 h, refresh token 30 days.
- Role-Based Access Control with 7 roles: super_admin, clinic_admin, doctor, assistant, receptionist, pharmacy, patient.
- User statuses: active, inactive, pending.
- First super admin seeded via CLI: flask create-super-admin.
- Passwords stored as hashes, never returned.

2.5 Auditing

An after_request hook auto-logs every successful mutating request (POST/PUT/PATCH/DELETE, status < 400) to the audit_log table with user_id, clinic_id, method, path, and status — excluding login. Failures roll back cleanly.

2.6 Configuration (env-driven)

config.py reads from .env with three profiles: development, production, testing (SQLite in-memory). DB URI: mysql+pymysql://user:pass@host:port/clinic?charset=utf8mb4. Secrets (SECRET_KEY, JWT_SECRET_KEY, DB_PASSWORD, OPENAI_API_KEY) come from environment.

3. Server / Runtime

Concern	Detail
Dev server	python run.py → Flask dev server, 0.0.0.0:5110
Prod server	gunicorn -w 4 -b 0.0.0.0:5110 "run:app" (4 worker processes)
TLS / domain	Served under clinic.nalexustechologies.com (reverse proxy in front of Gunicorn)
Database	MySQL 8, database clinic, user clinic, charset utf8mb4
Schema mgmt	Flask-Migrate (flask db upgrade) or raw migrations/schema.sql
Seed data	scripts/seed_demo_data.py

4. Frontend Architecture

4.1 Stack

Item	Detail
Framework	Flutter (Dart SDK >=3.3.0 <4.0.0)
Target	Web build (web/io conditional imports) + cross-platform capable
State management	Provider (ChangeNotifierProvider wrapping AuthService)
HTTP	http ^1.2.1 via a central ApiService
Token storage	flutter_secure_storage ^9.2.2 (web/io split)
UI	Material Design, google_fonts, custom theme (glassmorphism)
PDF/Export	pdf ^3.11.3 + export_service (PDF / CSV / JSON)
Date/format	intl ^0.19.0

4.2 Structure (mirrors backend domains)

```
lib/main.dart → bootstraps Provider, wires 401→login redirect
lib/app.dart → MaterialApp, named routes, route guards, global navigatorKey
lib/core/ → constants (api endpoints), services (one client per domain), theme, utils, widgets
lib/models/ → Dart data models
lib/routes/ → app_routes, route_guard (role-gated navigation)
lib/screens/ → per-role portals: super_admin / clinic_admin / doctor / assistant / receptionist /
pharmacy / patient / reports / auth
```

4.3 Key client behaviors

- ApiService centralizes HTTP + auth-header injection; a global setUnauthorizedCallback forces logout to login on any 401.
- route_guard.dart enforces RBAC client-side: reads currentUser.role and redirects to the role's dashboard if the route is not in allowedRoles.
- Conditional io vs web implementations for storage and export.

5. Models & AI — Inputs → Outputs

The system contains two distinct AI subsystems, both routed under `/api/clinical-ai/*`.

5.1 Disease Risk Models (scikit-learn, in-process)

Four supervised classifiers, trained once at first use (`lru_cache`) on a deterministic educational/demo dataset (numpy seed 42; breast cancer uses the real sklearn Wisconsin dataset). 75/25 stratified split; each reports test accuracy.

Model	Algorithm	Input features (ranges)	Output
diabetes	LogisticRegression (StandardScaler pipeline)	age (1–120), BMI (10–70), glucose (40–400 mg/dL), systolic BP (60–260), family history (0/1)	risk probability + percent + level
heart_disease	RandomForest (160 trees, depth 7)	age (18–120), sex (0/1), cholesterol (80–700), resting BP (60–260), max heart rate (40–240), smoker (0/1)	risk probability + percent + level
stroke	DecisionTree (depth 6)	age, hypertension (0/1), known heart disease (0/1), avg glucose (40–400), BMI (10–70), smoker (0/1)	risk probability + percent + level
breast_cancer	RandomForest (180 trees, depth 8)	mean radius, mean texture, mean perimeter, mean area, mean concavity	malignant-risk probability + percent + level

Output JSON: { model{key,name,algorithm,accuracy,features,safety_notice}, risk_probability (0–1), risk_percent (0–100), risk_level (low<0.35 / moderate<0.70 / high≥0.70), inputs, safety_notice }. Every feature is validated numeric and within min/max. Endpoints: GET `/risk-models`, POST `/risk-predict/<modelKey>`.

These are educational demonstrations only — explicitly not for diagnosis. Accuracy reflects synthetic demo data, not validated clinical performance.

5.2 Clinical AI (LLM) documentation assistant

Item	Detail
Provider (configurable)	CLINICAL_AI_PROVIDER = openai (default) or local
Model	CLINICAL_AI_MODEL = gpt-5.2 (env-overridable)
API	OpenAI Responses API (POST <code>/v1/responses</code>) with strict <code>json_schema</code> output
Timeout	45 s (configurable)
Fallback	If no API key or call fails → local rule-based engine (regex entity extraction + red-flag keyword rules) so it degrades gracefully offline
Provider auth	OPENAI_API_KEY from environment (never in reports)

Capabilities (inputs → output):

Endpoint	Input	What it does
POST <code>/patient-summary</code>	clinic/doctor/patient ids → patient profile (vitals, prescriptions, reports, appointments, follow-ups)	Concise clinician-facing history summary
POST <code>/consultation-assist</code>	appointment + patient ids + working note (symptoms, diagnosis, notes, vitals)	Draft symptoms/diagnosis/notes, entity extraction, red flags, patient-friendly explanation
POST <code>/extract-medical-text</code>	free-text string	Extract medical entities + short summary

Endpoint	Input	What it does
GET /status	—	Reports enabled / provider / model / features

Structured output (every response): summary, symptoms_draft, diagnosis_draft, notes_draft, patient_friendly_explanation, red_flags[], suggested_questions[], extracted_entities{conditions, medicines, dosages, symptoms, procedures, body_parts, lab_tests}, follow_up_guidance, provider, safety_notice.

Safety: a system prompt + safety_notice on every response enforce that output is a draft for a licensed clinician to verify — no autonomous diagnosis, prescription, or triage.

6. Reports & Exports

Backend report endpoints: clinic-revenue, doctor-revenue, pharmacy-sales, patient-visits, appointments, payments (/api/reports/*). The Flutter export_service renders these to PDF (via the pdf package), CSV, and JSON, with web/native code paths.

7. Evaluation Quick-Reference

Question	Answer
Domain deployed?	clinic.nalexustechologies.com (HTTPS); API at /api; internal 127.0.0.1:5110
Backend architecture?	Flask 3 REST API, App-Factory + 18 Blueprints, layered routes→services→models, JWT RBAC (7 roles), audit hook, env config
Which server?	Gunicorn (4 workers, port 5110) on Ubuntu/Linux behind a TLS reverse proxy; MySQL 8 database
Frontend architecture?	Flutter (Dart 3.3+) web app, Provider state mgmt, central ApiService + secure token storage, role-guarded routes, per-role portals
Models used?	4 scikit-learn classifiers (LogisticRegression, 2× RandomForest, DecisionTree) for educational risk + OpenAI gpt-5.2 LLM (local rule-based fallback)
Inputs → outputs?	Risk models: numeric clinical features → risk probability/percent/level. Clinical LLM: patient record / consultation note / free text → structured JSON drafts, entities, red flags, summaries

Prepared from direct inspection of clinic_backend (Flask) and clinic_app (Flutter) source on 16 June 2026.